Beyond the GPU

A Microarchitectural Analysis of LLM Tool-Calling Kernels

Kunpeng Zhang / Fazal Rehman / TJ Grewal

CMPT 450

What happens when the CPU becomes the bottleneck for AI?

Introduction & Motivation

The Paradigm Shift

LLMs are transitioning from **passive text generators** to **active, autonomous agents** utilized in complex engineering workflows.

The Misplaced Focus

Historically, optimization efforts have overwhelmingly targeted the **GPU forward-pass**, minimizing Time-To-First-Token (TTFT).

The New Bottleneck

As tool-calling expands, the execution bottleneck shifts abruptly **from the hardware accelerator back to the host CPU**.

The CPU is the new frontier.

Architectural Divergence

GPU Hostility

Agentic tool tasks behave more like traditional datacenter **Remote Procedure Calls (RPC)** than highly parallel machine learning kernels.

Scalar by Nature

Operations like parsing JSON, traversing filesystems, and querying databases are **inherently scalar**.

The Hardware Penalty

These tasks require deep recursive logic and complex memory pointer chasing, leading to:

- Massive **Last-Level Cache (LLC)** miss rates
- High **branch mispredictions**
- Frequent **pipeline stalls**

Agent tools are fundamentally hostile to GPU architectures.

Core Research Questions

RQ1 — Quantification

What is the macro-level "Agentic Tax" (latency penalty) imposed strictly by the LLM orchestration framework?

RQ2 — Microarchitectural Profiling

How do specific tool classes map to their defining hardware bottlenecks during multi-step loops?

RQ3 — Optimization Viability

Do classical, kernel-level hardware optimizations effectively propagate to end-to-end system latency, or are they absorbed by the orchestration layer?

Methodology & System Architecture

Decoupled Harness

Custom benchmarking suite that **strictly isolates** the Python/FastAPI orchestrator from compiled, native C++ execution kernels.

The Model

Qwen-2.5-7B-Instruct running locally in FP16 with sampling disabled to guarantee **deterministic ReAct cycles**.

Cycle-Accurate Telemetry

Integrated the Linux PMU via **PAPI** to capture strictly scoped hardware events exclusively during CPU execution windows:

- Instructions Per Cycle (IPC)
- Branch Mispredictions
- L3 Cache Misses
- Total Cycles

Testbed: Intel Xeon Gold 6242R @ 3.10 GHz — isolated, predictable cycle-to-latency.

Tool Kernels & Control Baseline

Compute-Bound

AST Evaluator

Nested math expressions stressing **branch prediction**.

Memory-Bound

SQLite Scan

Forced full-table scans intentionally thrashing the **LLC**.

I/O-Bound

FS Walker

Recursive traversal stressing **OS context switches**.

Control

Direct Pipeline

Static regex routing, **completely bypassing** the GPU & LLM.

The "Speed of Light" control isolates the exact cost of orchestration.

Key Finding 1 — The “Agentic Tax”

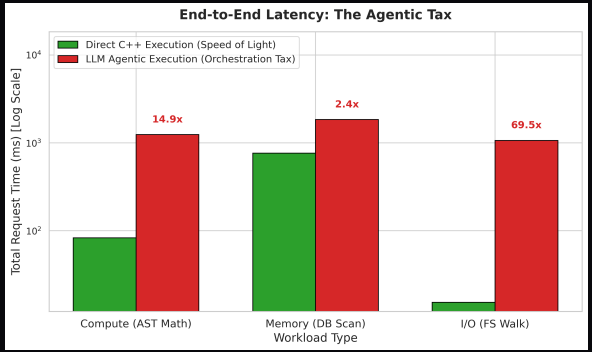

Figure 1: Log-scale latency — Direct C++ vs. Full Agentic Loop.

- **I/O Walk:** Native ∼15 ms → Agentic ∼1 s
  69.5× slowdown
- **AST Math:** **14.9**× penalty
- **DB Scan:** only **2.4**× (∼764 ms native)

Amdahl’s Law

The DB scan’s 2.5 billion native cycles amortize the fixed framework cost. The tax is a **massive fixed cost**, not a proportional scalar.

Key Finding 2 — Context Accumulation (Multi-Step)

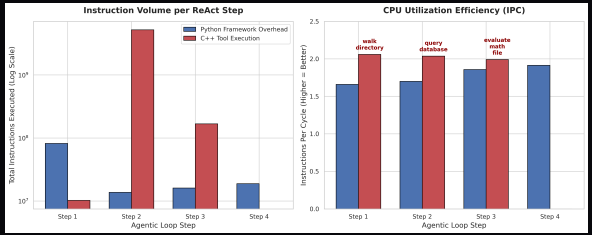

Figure 2: Instruction volume & IPC across a multi-step ReAct loop.

The Cold Start Penalty

At Step 1, the framework executes $\sim$**82 M instructions** simply to parse and load JSON schema definitions.

The Stateless Trap

Because the LLM is stateless, the framework's instruction volume **grows linearly** with each successive step, continuously re-serializing an expanding conversation history.

Key Finding 3 — The Masking Effect

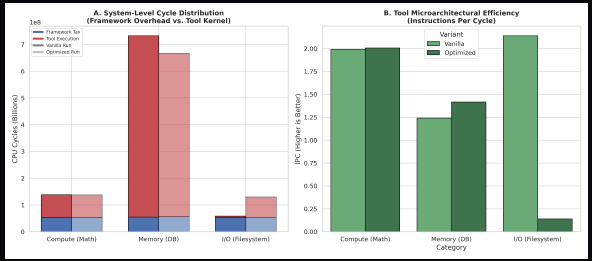

Figure 5: System-level cycle distribution across all optimized domains.

The Micro Win

Software prefetching (`__builtin_prefetch`) on the memory-bound DB kernel:

- Kernel IPC **+14.2%**
- Saved ∼**66 M** core CPU cycles

The System Reality

End-to-end system speedup remained **below 1%**.

The massive Python orchestration bloat **statistically swallows and nullifies** kernel-level microarchitectural victories.

Conclusion & Future Work

The Verdict

Current agentic architectures are **critically crippled** by the orchestration bottleneck. Hardware-software co-design must shift away from optimizing tool-execution kernels and **aggressively target the orchestration layer itself**.

Proposed Solutions

1. **Compiled Orchestrators**
   Transitioning to Rust/C++.
2. **Continuous Batching Engines**
   Adopting vLLM + PagedAttention.
3. **Binary Protocols**
   Replacing JSON with Apache Arrow.

Until the orchestration layer is treated as a high-performance data plane, the promise of agentic AI will remain masked by its own framework.

Live Demo & Q & A

Beyond the GPU:
A Microarchitectural Analysis of LLM Tool-Calling Kernels

Terminal 1: Run the Orchestrator (`model_loader.py`)

Terminal 2: Execute the Stress Payloads

Kunpeng Zhang · Fazal Rehman · TJ Grewal
CMPT 450

github.com/AndyZzzZzzZzz/beyond-gpu-kernels